

seL4 Timer Drivers and TimeServer

Page Content

- [Overview](#)
- [seL4_test](#)
- [Timer Driver](#)

Overview

The following seL4 libraries offer timer functionality (to user land):

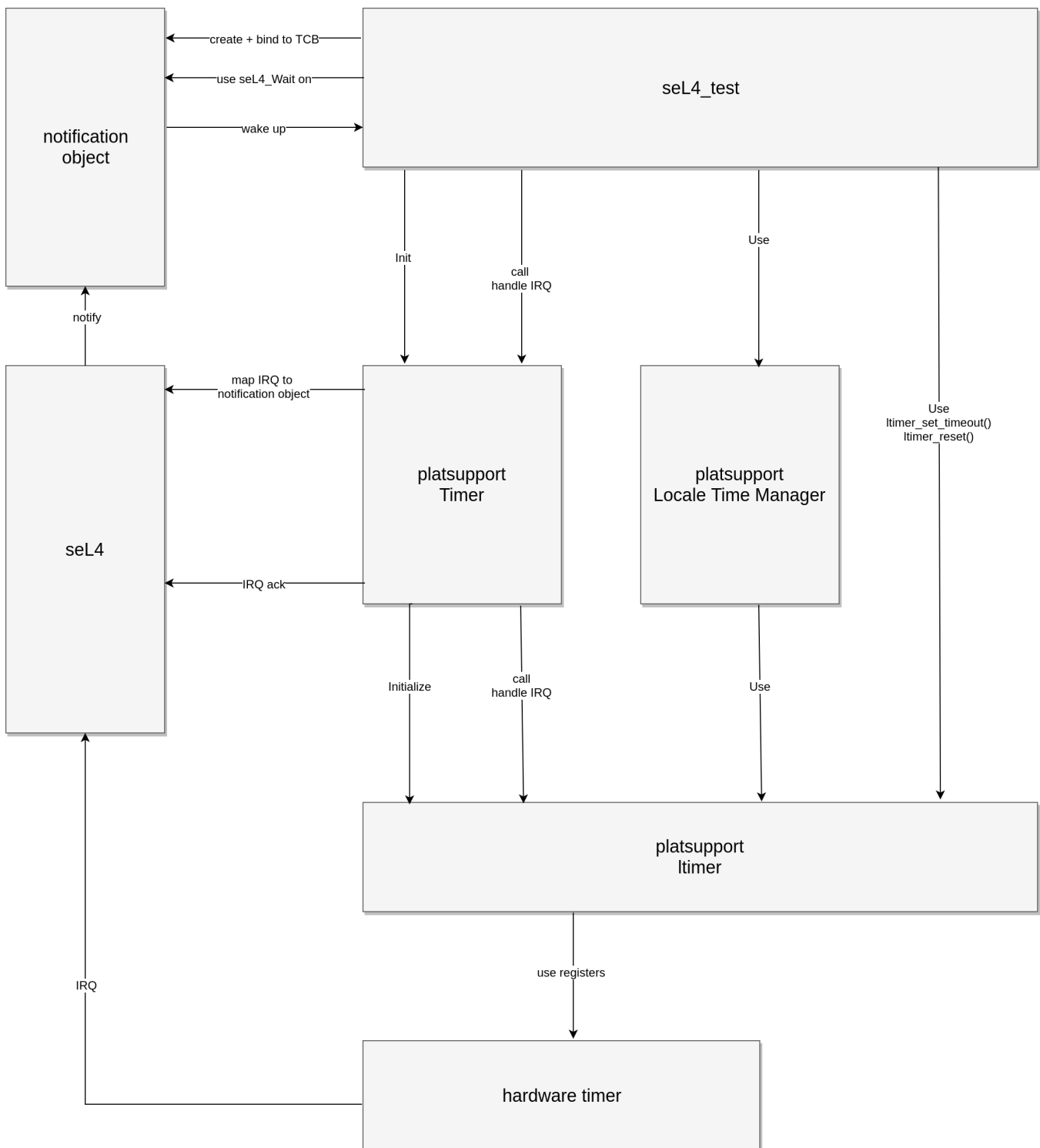
- `seL4_libs/libseL4platsupport/include/seL4platsupport/timer.h` is a fairly low level library which interfaces a seL4 timer driver to seL4. It contains the bare minimum functionality required to enable an application make use of a timer driver.
- `util_libs/libplatsupport/include/platsupport/local_time_manager.h` provides the ability of setting multiple timeouts. Internally it uses a timer driver.

These two libraries represent a low level and a high level way of using timers:

- `timer.h`: the application has to use the timer driver (`util_libs/libplatsupport/include/platsupport/timer.h`) directly for using timer functionality
- `local_time_manager.h`: the application can use the high level services of `util_libs/libplatsupport/include/platsupport/time_manager.h`.

seL4_test

`seL4_test` is actually using both ways of using timer functionality:



Summary of seL4_test using timer.h:

- create a notification object and map it to seL4_tests driver thread
- initialize an timer.h passing it the notification object
- timer.h tells seL4 to trigger the notification object when a timer interrupt occurs
- seL4_test uses the timer driver (ltimer.h) directly for timer functionality subsequently it waits on the notification object
- ltimer.h sets up the hardware timer accordingly
- the resulting interrupts will be handled by seL4
- seL4 as a result triggers the notification object

- when `seL4_test` wakes up because of this it calls `timer.h` to handle the interrupt
- `timer.h` acknowledges this interrupt with `seL4` and forwards this interrupt to be handled to the timer driver

Summary of `seL4_test` using `timer.h`:

- initialize a object of type `time_manager.h`
- use the high level timer functionality of `time_manager.h`

Timer Driver

A `seL4` timer driver is a software module implementing the `seL4` interface `util_libs/libplatsupport/include/platsupport/ltimer.h`. It uses, typically, two basic hardware timers of a target in order to provide the services defined by `ltimer.h`.

The methods of `ltimer.h` can, conceptually, be grouped like this:

- **query resources:**
 - `ltimer_get_num_irqs()`,
`ltimer_get_nth_irq()`,
`ltimer_get_num_pmems()`,
`ltimer_get_nth_pmem()`:
provide information about the resources of the used hardware timers: interrupts and regions of physical memory containing the memory mapped registers.
- **lifetime management:**
 - `ltimer_default_describe()`,
`ltimer_default_init()`:
are used to initialize an object of type `ltimer.h` ("`ltimer`"). Initialization includes operations like allocating memory (for internal data structures) as well as mapping the memory mapped regions of the used hardware timers. Access to these operations is given to an `ltimer` by passing it an object of type `ps_io_ops_t`. The `seL4` library function `seL4platsupport_new_io_ops` can be used to create such an object.
 - `ltimer_destroy()`:
stops the timer, un-maps the hardware timers and frees any allocated memory.
- **timer functionality:**
 - `ltimer_get_time()`:
read the current time in ns
 - `ltimer_get_resolution()`:
get the precision of this time returned by `get_time`
 - `ltimer_set_timeout()`:
start a timer of type periodic, absolute or relative.
 - `ltimer_handle_irq()`:
setting a timeout eventually results in interrupts handed down by `seL4` to some user land handler. This handler has to call `ltimer_handle_irq()` for handling the interrupt.
 - `ltimer_reset()`:
reset the timer into a state similar to what it was when first initialized.

Currently there exist about 12 different timer driver implementations for various platforms (search for `ltimer.c` in `util_libs/libplatsupport/src`). Looking at their implementation shows not quite a homogeneous but a recurring pattern:

- a timer driver implementation consists of two clearly separate parts:
 1. a low level part which consists of a driver implementation of a simple low level hardware timer
 2. a high level part which implements the `ltimer.h` functionality using the low level timer driver
- the interfaces of the low level parts of the various platforms are so similar that the corresponding high level parts differ only very little
- the way hardware timers are represented with memory mapped registers are so similar across platforms that even some of the low level parts differ only very little across platforms
- there is an implicit assumption which defines the implementation of a low level part: it knows the high level part will use two hardware timers where the first one is used as an ordinary timer (= programming timeouts) and the second one will be used as a long running timer thereby providing a time reference